



SecretSifter

Burp Suite Secret Detection Extension

Passive and active detection of exposed API keys, credentials, JWTs, PII, and connection strings in HTTP traffic — using 100+ anchored vendor rules, entropy heuristics, and a Bulk Scan engine with HTML and CSV reporting.

VERSION

1.2.x

PLATFORM

Burp Suite 2024.7+

AUTHOR

Hemanth Gorijala

LICENSE

MIT

Table of Contents

- 1. Executive Summary**
- 2. Problems SecretSifter Solves**
- 3. Features Overview**
 - 3.1 Passive Scanning
 - 3.2 Bulk Scan
 - 3.3 HAR File Import
 - 3.4 Scan Tiers
 - 3.5 Detection Rule Summary
 - 3.6 AI Triage
- 4. Usage Guide**
 - 4.1 Installation
 - 4.2 Passive Scanning (Automatic)
 - 4.3 Bulk Scan Workflow
 - 4.4 Right-click Rescan
 - 4.5 Settings Tab
 - 4.6 Reports & Export
- 5. Detection Engine Deep Dive**
 - 5.1 Scan Pipeline
 - 5.2 Anchored Vendor Token Rules (100+)
 - 5.3 Context-Gated Rules (40+)
 - 5.4 False Positive Reduction
- 6. Severity & Confidence Model**
- 7. Triage Guide & Error Cases**
- 8. Technical Architecture**
- 9. Requirements & Compatibility**
- 10. Building from Source**

11. Credits & License

1. Executive Summary

What SecretSifter does and why it matters.

SecretSifter is a Burp Suite extension for passive and active detection of secrets exposed in HTTP traffic. It integrates directly into the Burp proxy pipeline — no configuration required — and begins reporting findings immediately after installation.

Modern web applications leak credentials in JavaScript bundles, configuration endpoints, API responses, and request headers. Manual review of this traffic is impractical at scale. SecretSifter automates the detection layer using a multi-phase scan engine: anchored vendor token patterns, Shannon entropy heuristics, context-gated rules, and PII validators — filtering out noise while surfacing real, actionable secrets.

Findings appear as AuditIssues in Burp's Dashboard (Pro) and in the Bulk Scan results table (all editions), with severity classification, matched value preview, and one-click HTML/CSV export for reporting.

Community Edition: Passive scan check (Dashboard → Issue Activity) requires Burp Suite Professional. All other features — Bulk Scan, right-click rescan, proxy handler, sitemap sweep, HAR import, and reports — work in Community Edition.

2. Problems SecretSifter Solves

The security problems this tool addresses.

Problem	How SecretSifter Solves It
Secrets in JS bundles API keys, tokens, and service credentials embedded in React/Vue/Angular bundles and shipped to every browser visitor.	Passive scan fires on every proxied response. Bulk Scan follows <code><script src></code> and webpack chunks. HTML inline script blocks are extracted and scanned separately.
Credentials in API responses Config endpoints (<code>/api/config</code> , <code>/__env</code>) returning keys in JSON that are not meant to be client-accessible.	JSON body walker traverses all key-value pairs at every depth. SSR state blobs (Next.js <code>__NEXT_DATA__</code> , Nuxt <code>__NUXT__</code>) are detected and unwrapped.
Credentials in request headers Custom authentication headers (<code>App_key</code> , <code>Ocp-Apim-Subscription-Key</code> , <code>Resource</code>) that are client-controlled and potentially over-privileged.	Request header scanning (opt-in) checks custom headers for high-value credential patterns across every proxied request.
PII in responses Social Security Numbers or credit card numbers appearing in API responses, error pages, or debug output.	PII scanner with Luhn validation (cards) and SSN format checks. Runs in FULL tier; toggle in Settings.
Dynamic SPA API calls missed by static scan XHR/Fetch calls only made after JavaScript executes — invisible in the site map until the page is actually visited.	Headless Browse launches Chrome/Chromium through Burp proxy to capture dynamic calls. Scope Monitor captures passive findings from browsed sessions and routes them into the Bulk Scan table.
Authenticated or offline content Responses captured in HAR files from authenticated sessions that cannot be replayed.	HAR import scans responses from <code>.har</code> files directly — no live HTTP fetch required.

High false positive rate

Entropy-based scanners produce noise on obfuscated CSS class names, UUIDs, and webpack hashes.

60+ noise key-name filters, Angular/Vue directive suppression, UUID rejection, JWT body suppression, Luhn validation, CDN blocklist, and user-configurable key name blocklist/allowlist.

3. Features Overview

Complete feature list with descriptions.

Feature	Detail
Passive scanning	Fires on every proxied response automatically; also sweeps existing sitemap on load.
100+ anchored token rules	GitHub, GitLab, AWS, Stripe, OpenAI, Slack, Shopify, Azure, GCP, Docker Hub, Clerk, and more.
40+ context-gated rules	Algolia, Cloudflare, Zendesk, Heroku, Datadog, Salesforce, Mistral, Cohere, Auth0, Supabase, and more.
Request header scanning	Detects credentials in custom headers (e.g. <code>App_key</code> , <code>Resource</code> , <code>Ocp-Apim-Subscription-Key</code>).
Generic KV & high-entropy scanner	Catches unlisted keys using keyword + entropy heuristics.
PII detection	SSN, credit card numbers (Luhn-validated), credential-bearing URLs.
Bulk Scan tab	Paste/import URL lists; follows <code><script src></code> , webpack chunks; 1–50 concurrent threads.
HAR import	Scan responses from a <code>.har</code> file directly — no live fetch needed (useful for auth-walled or offline targets).
Headless Browse	Optionally launch Chrome/Chromium headless through Burp proxy to capture dynamic XHR/Fetch calls.
Scope Monitor	Capture passive proxy findings for watched hosts and route them into the Bulk Scan results table.
HTML reports	Per-scan all-in-one HTML report; per-domain ZIP (one file per hostname); CSV export.
Scan tiers	FAST / LIGHT / FULL — trade speed vs. coverage.
Key name blocklist / allowlist	Suppress FP-prone key patterns or force-report specific key names regardless of entropy.
Inline response tab	"SecretSifter" tab in Proxy History / Repeater showing findings for the current response.

AI Triage

Per-finding and bulk "AI Triage All" using Burp's built-in AI (Pro + AI features enabled).

3.1 Passive Scanning

Once loaded, SecretSifter hooks into the Burp proxy pipeline and scans every response body that passes through the proxy. The proxy handler fires on all traffic — not scope-limited — so secrets in third-party API calls are also captured.

On extension load, a background thread sweeps all responses already present in Burp's Target → Site Map, so findings appear immediately even for traffic captured before the extension was installed. This sweep runs once per session on a daemon thread and exits cleanly on unload.

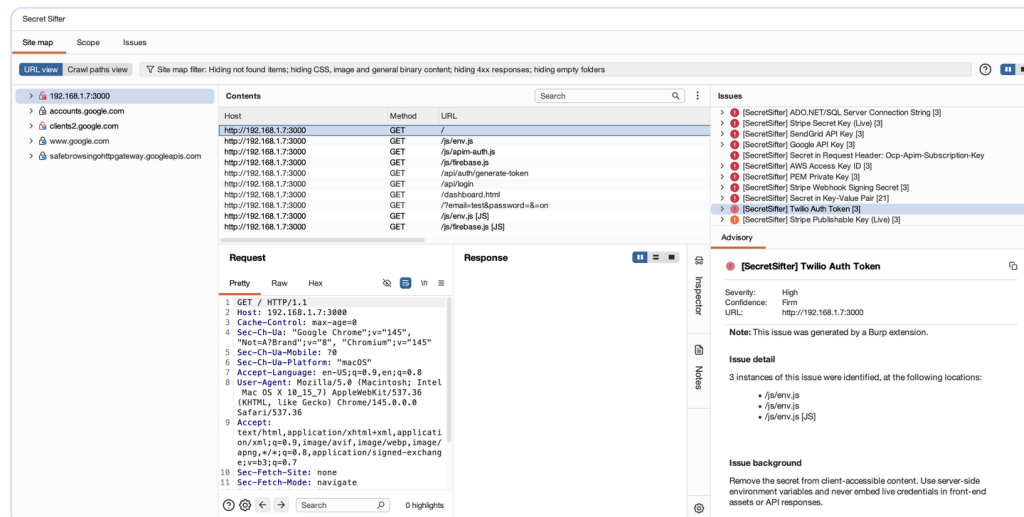


Figure 1 — Dashboard → Issue Activity: SecretSifter findings grouped by rule, with severity badges and source URLs.

3.2 Bulk Scan

The Bulk Scan tab accepts a list of target URLs and fetches them concurrently (1–50 threads). For each URL it can follow `<script src>` references and webpack chunk manifests (depth 1), optionally launching a headless Chrome instance to capture dynamic XHR/Fetch calls that only fire after JavaScript executes.

3.4 Scan Tiers

Tier	Rules Active	Use When
FAST	100+ anchored vendor tokens only	Quick recon, large site maps, passive scan on high-traffic sites.
LIGHT	+ High-entropy scanner + 40+ context-gated rules + DB connection strings	Standard penetration test — good balance of speed and coverage.
FULL	+ PII (SSN, CC) + Generic KV + SSR state blobs + JSON walker + getter functions	Deep audit, bug bounty, maximum coverage.

3.5 Detection Rule Summary

Category	Count	Examples
Anchored vendor tokens	100+	GitHub PAT, GitLab token, AWS Access Key, Stripe sk_, OpenAI sk-, Slack xoxb/xapp, Shopify shpss_, Azure storage key
Context-gated rules	40+	Algolia API key (near "algolia"), Cloudflare token, Zendesk token, Heroku key, Datadog key, Salesforce token, Auth0 client secret
DB connection strings	10+	PostgreSQL, MySQL, MongoDB, Redis, MSSQL — full URI with embedded credentials
PII	3	US SSN (XXX-XX-XXXX), credit card (Luhn-validated, 13–19 digits), credential-bearing URLs
Generic entropy / KV	Heuristic	Any key-value pair where the key looks credential-like and the value has entropy $\geq$ threshold
SSR state blobs	5+	<code>__NEXT_DATA__</code> , <code>__NUXT__</code> , <code>window.__INITIAL_STATE__</code> , <code>REDUX_STATE</code>

3.6 AI Triage (Pro + AI)

SecretSifter integrates with Burp Suite's built-in AI features (Professional edition, AI features enabled). Two modes are available:

- **Per-finding triage:** Right-click any finding → AI Triage. Burp's AI evaluates the finding context and returns a confidence assessment with an explanation.
- **AI Triage All:** One-click button in the Bulk Scan tab to triage all findings in the current session. Results are appended to each finding row.

AI Triage requires Burp Suite Professional with AI features enabled in Burp Settings → AI. The feature is unavailable in Community Edition and in any edition where AI features are disabled.

4. Usage Guide

Step-by-step instructions for all scan modes.

4.1 Installation

Step 1 — Choose Your JAR

JAR	Entry Point	Use When
<code>secretsifter-store-*.jar</code>	<code>SecretScannerExtensionStore</code>	Standard use, BApp Store install. No embedded HTTP server.
<code>secretsifter-mcp-*.jar</code>	<code>SecretScannerExtension</code>	Claude Code / MCP agent integration. Starts JSON-RPC server on <code>127.0.0.1:8765</code> .

Step 2 — Load into Burp Suite

1. Open Burp Suite → **Extensions** tab → **Installed** → **Add**
2. Set **Extension type**: Java
3. Browse to your chosen JAR file
4. Click **Next** — the extension loads and a **Secret Sifter** tab appears
5. Check **Extensions** → **Output** for the version confirmation line:

`[*] Loaded: SecretSifter v1.2.x (Store)`

4.2 Passive Scanning (Automatic)

No action required. Once loaded, the extension scans all responses flowing through the Burp proxy. Findings appear in:

- **Dashboard** → **Issue Activity** — as Burp AuditIssues (Pro only)
- **Target** → **Site Map** — under the target host with a SecretSifter issue icon
- **Secret Sifter** → **Bulk Scan** results table (all editions)

- **SecretSifter tab** in Proxy History / Repeater — findings for the selected response

4.3 Bulk Scan Workflow

1. Navigate to **Secret Sifter** → **Bulk Scan**
2. Paste one URL per line, or import a `.txt` / `.csv` / `.har` file
3. Configure scan options (see table below)
4. Click ► **Start Scan**
5. Findings populate the table in real time, sorted by severity
6. Click **Export** → HTML Report, HTML Report (per domain), or CSV

Bulk Scan Options Reference

Option	Default	Description
Tier	FULL	FAST / LIGHT / FULL — controls which scan phases run
Threads	25	1–50 concurrent URL workers
Proxy port	8080	Burp proxy port for headless browse routing
Follow <script src>	On	Fetch and scan all referenced JS files found in HTML
Follow webpack chunks	On	Follow chunk references inside JS bundles (depth 1)
Scope Monitor	On	Capture passive proxy findings for watched hosts
Cross-origin APIs	On	Capture XHR/Fetch calls fired to other domains from watched hosts
Headless Browse	Off	Launch Chrome headless through Burp proxy to capture dynamic JS API calls. Opt-in; all traffic routed through local proxy only — no data leaves your machine.
Debug	Off	Log each scanned URL to Extensions → Output
Scan Site Map	Button	Sweep all JS/HTML responses already captured in Burp's Target site map

4.4 Right-click Rescan

In **Proxy → HTTP History** or **Repeater**, right-click any request and choose **Rescan for Secrets**. The context menu expands to all site-map entries for the selected host(s). Results open in the Bulk Scan panel. An HTML report can be saved after the scan completes.

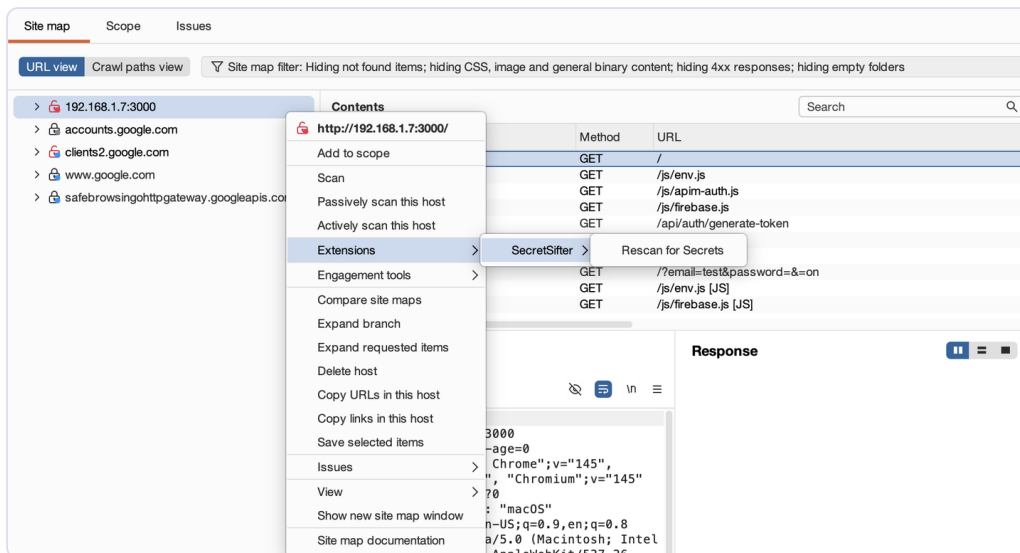


Figure 3 — Context menu in Proxy History: "Rescan for Secrets" scans all site-map entries for the selected host.

4.5 Settings Tab

Setting	Description
Scan Tier	FAST / LIGHT / FULL
Entropy Threshold	Minimum Shannon entropy for high-entropy scanner (default: 3.5 bits/char). Increase to 4.0–4.5 if getting too many FPs.
PII Detection	Enable/disable SSN and credit card scanning
Scan request headers	Scan custom request headers (e.g. <code>App_key</code> , <code>Resource</code>) for credentials
CDN Blocklist	Hostnames to skip; pre-populated with common CDN/ analytics domains. One per line.
Key Name Blocklist	Substring patterns — matching key names are suppressed (e.g. <code>STATE_KEY_</code> , <code>NEXT_PUBLIC_</code>)
Key Name Allowlist	Substring patterns — matching key names are always reported regardless of entropy
MCP Bridge	Enable/disable the JSON-RPC server for Claude Code integration (mcp build only)

Secret Sifter

Bulk Scan

Settings

Scanner Controls

☒ Enable passive scanning

Scan Tier:

FULL

Entropy threshold:

3.5

☒ Enable PII detection (SSN, Credit Cards)

☒ Scan request headers / body for secrets

CDN / Third-Party Blocklist (one entry per line — partial host match)

cdnjs
jsdelivr
unpkg
ajax.googleapis.com
ajax.microsoft.com
code.jquery.com
stackpath

Key Name Blocklist (suppress — one pattern per line, substring match)

STORAGE_KEY_
storage_key_
STATE_KEY_
state_key_
NEXT_PUBLIC_
REACT_APP_PUBLIC_
VUE_APP_PUBLIC_

Key Name Allowlist (force-report — one pattern per line, substring match)

apiKey

Save Settings

Reset to Defaults

Settings loaded from preferences.

Figure 4 — Settings Tab: scan tier, entropy threshold, PII toggle, CDN blocklist, and key name filters.

4.6 Reports & Export

SecretSifter supports three export formats, all available from the Bulk Scan panel toolbar:

Format	Description
HTML Report	Single self-contained HTML file with all findings, severity badges, matched value preview (masked), source URLs, and context snippets. Suitable for client delivery.
HTML Report (per domain)	ZIP archive with one HTML file per hostname. Useful when a scan covered many targets and you want to deliver per-client reports.
CSV	Flat spreadsheet: severity, rule name, matched value, source URL, timestamp. Import into Jira, Notion, or any ticketing system.

Secret Sifter — Findings Report

Generated: 2020-03-20 18:36:59 | Target: http://162.168.17.3000/ | Mode: FULL | Total Findings: 21

Filter

AllHighMediumLowInfo

Domains: All domains

Rule: All rules

Search key / value / URL...

CSVJSON

Value

URL

#	Severity	Confidence	Rule ID	Key	Value	URL	Line	Context
1	HIGH	CERTAIN	DB_CONN_OUTNET	db_password	Pr5d8g12041	http://162.168.17.3000/ [HTML]	12	m / :insureh1d182024 (db_conn) Server-prod db-insureh1d182024 (internal:Database-insureh1d182024) (Server ID=sa/Passw...
2	HIGH	CERTAIN	GOOGLE_KEY_001	google_api_key	AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR	http://162.168.17.3000/firebase.js [JS]	7	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
3	HIGH	CERTAIN	PEM_PRIVATE_KEY_001	pem_private_key	-----BEGIN RSA PRIVATE KEY-----	http://162.168.17.3000/firebase.js [JS]	22	-----BEGIN RSA PRIVATE KEY-----
4	LOW	CERTAIN	GCP_KEY_001	gcp_sa_client_email	firebase-adminsdk@insureh1d182024-prod.iam.gserviceaccount.com	http://162.168.17.3000/firebase.js [JS]	23	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
5	HIGH	CERTAIN	GENERIC_KEY	apiKey	AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR	http://162.168.17.3000/firebase.js [JS]	7	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
6	LOW	CERTAIN	GENERIC_KEY	appId	1:481035492714:web:3f8a2b1c4d5e67a8b3b6	http://162.168.17.3000/firebase.js [JS]	13	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
7	HIGH	CERTAIN	GENERIC_KEY	private_key_id	key-id-9f3a8b2c1d4e5f6a7b8c9d0e	http://162.168.17.3000/firebase.js [JS]	21	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
8	LOW	FIRM	ENTROPY_TOKEN	client_id	100834268850647983421	http://162.168.17.3000/firebase.js [JS]	24	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
9	MEDIUM	CERTAIN	GENERIC_KEY	serviceaccountkey	svc-key-7e4b2a1f9c3d6e8a0b5f2c4d7e1a3b6	http://162.168.17.3000/ [HTML]	24	firebaseConfig = { apiKey: "AIzaSyO9t3ckeT2Smb9Xq6gQwPy8T3h7k8RcR", authDomain: "insureh1d182024.firebaseapp.com", databaseURL: "https://insureh1d182024.firebaseio.com", projectId: "insureh1d182024", storageBucket: "insureh1d182024.appspot.com", messagingSenderId: "481035492714", appId: "1:481035492714:web:3f8a2b1c4d5e67a8b3b6", measurementId: "G-481035492714"}
10	HIGH	CERTAIN	STRIPPED_KEY_001	stripe_secret_key	sk_live_31Q7m9K3j2w8T1c0R8eF9g4Kz2V6k6UPJH	http://162.168.17.3000/ [HTML]	12	stripeSecretKey: "sk_live_31Q7m9K3j2w8T1c0R8eF9g4Kz2V6k6UPJH", stripePublicKey: "pk_live_31Q7m9K3j2w8T1c0R8eF9g4Kz2V6k6UPJH", // Email delivery id ...
11	HIGH	CERTAIN	SENDGRID_KEY_001	sendgrid_api_key	SG.ck8Wnc1W7y1779b8aYm9.7p88naT6aKfvg7551G8	http://162.168.17.3000/ [HTML]	15	sendgridApiKey: "SG.ck8Wnc1W7y1779b8aYm9.7p88naT6aKfvg7551G8", // Email delivery id ...

Figure 5 — HTML Report: findings grouped by severity with masked value preview and context excerpts.

5. Detection Engine Deep Dive

How the scan pipeline works under the hood.

5.1 Scan Pipeline

For each response body, SecretSifter runs a multi-phase pipeline. Phases run in order; each phase can be toggled by scan tier:

```
Input: response body (string) + Content-Type + source URL
|
1. Content-type gate – skip binary, image, video, audio, font
   responses
2. HTML inline script extraction – extract text content of
   <script> blocks
3. Anchored token scan (FAST+) – vendor-specific regex against
   raw body
4. Context-gated scan (LIGHT+) – keyword proximity check then
   regex
5. DB connection string scan (LIGHT+) – URI pattern with
   embedded credentials
6. High-entropy KV scan (LIGHT+) – key=value pairs with entropy
   ≥ threshold
7. SSR state blob unwrap (FULL) – parse JSON blobs, then re-run
   pipeline
8. JSON walker (FULL) – recursive key-value traversal of JSON
   bodies
9. PII scan (FULL) – SSN, credit card, credential URLs
10. Getter function scan (FULL) – return "secret_value" patterns
    in JS
|
Output: List<SecretFinding>
```

5.2 Anchored Vendor Token Rules (100+)

Anchored rules use vendor-documented format specifications. Each rule has a regex with a fixed prefix that uniquely identifies the vendor (e.g. `ghp_` for GitHub PATs, `sk-` for OpenAI keys). This makes the pattern highly specific with near-zero false positive rate.

Vendor	Token Type	Pattern Prefix
GitHub	Personal Access Token	ghp_ , gho_ , ghu_ , ghs_ , ghr_
GitLab	Personal / Project token	glpat- , glptt-
AWS	Access Key ID	AKIA , ASIA , AR0A
Stripe	Secret / Restricted key	sk_live_ , rk_live_
OpenAI	API key	sk-proj- , sk-
Slack	Bot / App token	xoxb- , xapp- , xoxe-
Shopify	Partner / Store token	shpss_ , shppa_ , shpat_
Azure	Storage / SAS key	Format-matched 64-char base64 or SAS token structure
GCP	Service account key ID	Embedded in JSON key files
Docker Hub	Access token	dckr_pat_
Clerk	Secret key	sk_live_ + Clerk-specific format
...and 90+ more		

5.3 Context-Gated Rules (40+)

Context-gated rules require a vendor keyword to appear in proximity (within ~200 characters) before the value pattern is triggered. This prevents the high entropy value from matching unrelated content.

Example: Algolia API key rule only triggers if `algolia` appears near a 32-character alphanumeric string. Without the proximity check, any 32-char string would match.

Vendors covered include: Algolia, Cloudflare, Zendesk, Heroku, Datadog, Salesforce, Mistral, Cohere, Auth0, Supabase, Pinecone, Twilio, SendGrid, Mailchimp, Mailgun, and more.

5.4 False Positive Reduction

Mitigation	What It Prevents
60+ noise key-name filter (<code>NOISE_KEYNAMES</code>)	Keys ending in <code>Type</code> , <code>Error</code> , <code>State</code> , <code>Version</code> , etc. — discriminator enums, not secrets.
UUID rejection	8-4-4-4-12 format UUIDs filtered out unless key name ends in <code>Secret</code> or <code>Token</code> .
Angular / Vue directive filter	Strings matching Angular expression or Vue directive syntax (<code>v-</code> , <code>ng-</code>) suppressed.
JWT body suppression	JWT payload segment (the middle <code>eyJ...</code> block) filtered — this is base64 JSON metadata, not a secret.
Encoding alphabet filter	Values composed entirely of characters common in CSS hex colors, base64 padding, or webpack hashes are suppressed.
CC floating-point guard	16-digit sequences that are clearly floating-point numbers (contain a decimal) are not reported as credit card numbers.
CDN blocklist	Responses from common CDN domains (jQuery, Bootstrap, Analytics) are skipped entirely.
Luhn validation	Credit card candidates must pass the Luhn checksum algorithm.
Key name blocklist (user-configurable)	Any key name matching a user-defined substring pattern is suppressed.
Key name allowlist (user-configurable)	Key names matching allowlist patterns bypass entropy thresholds and are always reported.

6. Severity & Confidence Model

How findings are scored and what each level means for triage.

6.1 Severity Levels

Level	Meaning	Typical Finding
HIGH	Confirmed active credential — rotate immediately.	GitHub PAT, AWS Access Key, Stripe live secret key, OpenAI API key.
MEDIUM	Token that confirms a live service integration.	Algolia search key, Cloudflare API token, Auth0 client secret.
LOW	Identifier or key with limited standalone risk.	Public bucket name, CDN zone ID, analytics measurement ID.
INFORMATION	Structural or schema-level finding.	DB connection string without credentials, JWT header.

6.2 Confidence Levels

Confidence	Basis
CERTAIN	Anchored rule with vendor-documented format — false positive is theoretically impossible.
FIRM	Context-gated rule or DB connection string — vendor keyword in proximity confirms intent.
TENTATIVE	Entropy heuristic or generic KV rule — possible false positive; verify manually.

7. Triage Guide & Error Cases

How to investigate, validate, and resolve common issues.

7.1 Validating a Finding

Severity / Confidence	Validation Steps
HIGH / CERTAIN	1. Check the source URL — is it a production endpoint? 2. Confirm the value appears in the live response (Burp Response tab in Advisory panel). 3. For API keys: attempt a minimal API call to verify the key is active (in scope of your engagement). 4. For AWS keys: run <code>aws sts get-caller-identity</code> to check if active. 5. Export the SecretSifter HTML report — it includes a masked value preview and context snippet.
MEDIUM / FIRM	1. Examine the key name and matched value in the finding detail. 2. Check the source URL path — <code>config.js</code> , <code>app-config.json</code> suggest real config. 3. Run AI Triage (Pro) — Burp's AI can confirm or downgrade the finding. 4. If the value looks like a vendor's own demo key, mark as false positive.
LOW / TENTATIVE	Verify the identifier is credential-bearing (e.g. an Algolia key that allows write queries). If it grants no sensitive access, document as informational.

7.2 Common Error Conditions

Symptom	Likely Cause	Resolution
Extension does not load	Burp version < 2024.7 or loading the plain compile JAR	Verify Burp ≥ 2024.7. Load <code>secretsifter-store-*.jar</code> or <code>secretsifter-mcp-*.jar</code> — not a plain class file. Check Extensions → Output .
No findings for a known secret	Scan tier too low, JS file not fetched, or traffic not flowing through proxy	1. Switch Settings → Scan Tier to FULL. 2. Enable Follow <script src> and Follow webpack chunks in Bulk Scan. 3. Browse the target through Burp Browser, then click Scan Site Map. 4. For headers: enable Scan request headers in Settings.
Burp slows during passive scan	FULL tier entropy scan on high-volume CDN traffic	Switch to FAST tier. Expand CDN blocklist. Disable PII scanning.
Bulk scan finds secrets but Target → Site Map shows nothing	Directly-fetched JS files have no site map entry; Burp "Hiding not found items" filter hides them	Browse the target through Burp Browser first — this seeds the site map root URL. SecretSifter then files issues at the root URL so they appear in the site map.
Headless Browse does nothing / Chrome not found	Chrome / Chromium not installed or not on PATH	Install Google Chrome or Chromium. On macOS, <code>/Applications/Google Chrome.app</code> is auto-detected. Check Extensions → Output for <code>[Headless] Chrome/Chromium not found</code> .
AI Triage All button greyed out	Community Edition, or AI features disabled	Go to Burp Settings → AI → uncheck Disable AI features. Reload the extension.

Findings in Bulk Scan but not in Dashboard	Dashboard → Issue Activity requires Burp Pro	All findings are in the Bulk Scan table and HTML/CSV export in both editions.
High false positive rate from entropy scanner	Entropy threshold too low, or target uses obfuscated keys	Increase entropy threshold to 4.0–4.5 in Settings. Add FP-prone key names to Key Name Blocklist.
Same finding reported multiple times	Extension reloaded mid-session; deduplicator cleared on reload	Deduplication is session-scoped. Each session deduplicates within itself. Reload Burp or unload/reload the extension to reset.
MCP Bridge won't start	Port 8765 in use, or using the store JAR instead of the MCP JAR	Ensure you loaded <code>secretsifter-mcp-*.jar</code> . Check that port 8765 is free (<code>lsof -i :8765</code>). Enable MCP Bridge in Settings → MCP Bridge.
MCP: no response from Claude after <code>claude mcp add</code>	Claude Code was not restarted after registering the MCP server	Exit Claude Code and reopen it (or start a new session). MCP servers are loaded only at session start — a running session won't pick up a newly registered server.

7.3 False Positive Determination Guide

Pattern	FP Indicator	Action
Value contains <code>example</code> , <code>YOUR_KEY</code> , <code>xxxx</code> , <code>changeme</code>	Documentation placeholder	Should be auto- filtered; if not, add key name to blocklist.
Source URL is a CDN (<code>cdn.jsdelivr.net</code> , <code>unpkg.com</code>)	Open-source library — vendor's own demo key	Add domain to CDN Blocklist in Settings.
Value is a valid UUID (8-4-4-4-12 format)	Azure tenant/client ID — not a secret	Auto-filtered unless key name ends in <code>Secret</code> or <code>Token</code> .
Key name ends in <code>Type</code> , <code>Error</code> , <code>State</code>	Enum / type discriminator	Auto-filtered by noise key-name list.
Source URL is <code>test.</code> , <code>dev.</code> , <code>staging.</code>	Non-production credential — lower risk	Document as out-of- scope risk; rotate if same key is used in production.

8. Technical Architecture

Component design and internal structure.

8.1 Tech Stack

Component	Detail
Language	Java 17
API	Burp Montoya API (2024.7+)
Build	Gradle 8 + Shadow JAR (fat JAR, ~430 KB)
JSON parsing	Gson 2.10.1 (Apache 2.0)
Tests	JUnit 5.10.2

8.2 Component Map

Class	Role
<code>SecretScannerExtension</code>	Full build entry point (MCP + Burp AI)
<code>SecretScannerExtensionStore</code>	Store build entry point (Burp AI only, no MCP)
<code>SecretScanner</code>	Stateless multi-phase scan engine
<code>Patterns</code>	All compiled regex patterns and rule metadata
<code>ScanSettings</code>	Persisted configuration model
<code>SecretFinding</code>	Immutable finding record + AuditIssue builder
<code>BulkScanPanel</code>	Bulk Scan UI, scan orchestration, report export
<code>SettingsPanel</code>	Settings UI with preference persistence
<code>SecretScanCheck</code>	Passive scan check (Pro only)
<code>SecretProxyHandler</code>	Proxy response handler — all traffic, cross-origin aware
<code>SecretContextMenu</code>	Right-click "Rescan for Secrets"
<code>SecretFinding</code>	Immutable finding record + AuditIssue builder
<code>SitemapDeduplicator</code>	Two-namespace ConcurrentHashMap dedup guard (bulk + proxy)
<code>ScopeMonitor</code>	Routes cross-origin proxy findings into Bulk Scan table
<code>HtmlReportGenerator</code>	HTML + per-domain ZIP report generator
<code>McpBridge</code>	JSON-RPC server for Claude Code / MCP integration (mcp build only)
<code>SecretSifterTab</code>	Inline HTTP response tab in Proxy History / Repeater

8.3 Initialization Sequence

```
initialize(MontoyaApi api)
|
1. ScanSettings           ← load from Burp Preferences
2. SecretScanner          ← stateless engine, injected with
settings
3. SecretScanCheck        ← register passive scan check (Pro
only; gracefully skipped)
4. SecretProxyHandler     ← register proxy response handler (all
traffic)
5. SecretContextMenu      ← register right-click menu provider
6. SecretSifterTab        ← register inline HTTP response tab
7. SettingsPanel +
   BulkScanPanel          ← register single "Secret Sifter" top-
level tab
8. loadFromPreferences()  ← sync persisted settings to UI
9. McpBridge.start()      ← optional; only if MCP enabled (mcp
build only)
10. SitemapSweep thread   ← daemon thread, starts after 1.5 s,
scans existing site map
11. registerUnloadingHandler ← clean shutdown of all threads/
executors
```

8.4 Four Scan Entry Points

Entry Point	How It Triggers	Scope
Passive scan check	Every proxied response, via Burp's scanner API	In-scope only (Burp Pro)
Proxy handler	Every proxy response, background thread	All traffic, cross-origin aware
Right-click rescan	User selects "Rescan for Secrets" in context menu	Selected host + full site map
Bulk scan	User pastes URL list, 1–50 concurrent threads	User-specified URLs + following

8.5 Thread Safety Model

- `SecretScanner` is **fully stateless** — safe for concurrent calls from any thread
- `ScanSettings` uses `volatile` fields for primitives; `synchronized` methods for list operations
- `seenRequestValues` (cross-request dedup) uses `Collections.synchronizedSet()`
- `currentTargetUrl` uses `ThreadLocal` ; `hostToTargetUrl` uses `ConcurrentHashMap`
- `SitemapDeduplicator` uses two `ConcurrentHashMap` instances — separate namespaces for bulk scan and proxy handler
- Proxy handler submits scan work to a fixed-size `ExecutorService` — the proxy pipeline is never blocked

9. Requirements & Compatibility

System requirements and edition compatibility.

Component	Requirement	Notes
Burp Suite Professional	2024.7+	Required for Dashboard Issue Activity and AI Triage. All other features available in Community.
Burp Suite Community	2024.7+	Bulk Scan, rescan, proxy handler, and sitemap sweep work. Passive scan check gracefully skipped.
Java	17+	Bundled with Burp Suite — no separate installation needed.
Chrome / Chromium	Any current version	Required only for Headless Browse. Auto-detected on macOS, Windows, Linux.
OS	macOS (Intel / Apple Silicon), Windows (x64), Linux	Cross-platform Chrome detection built in.

9.1 Feature Matrix by Edition

Feature	Community Edition	Professional
Passive scan check (Dashboard / Issue Activity)	✗	✓
Proxy handler (all traffic scanning)	✓	✓
Bulk Scan tab	✓	✓
Right-click rescan + HTML report	✓	✓
Sitemap sweep on load	✓	✓
HAR import	✓	✓
Headless Browse	✓	✓
AI Triage (per-finding + bulk)	✗	✓ (AI features enabled)
CSV + HTML report export	✓	✓
MCP Bridge (mcp build only)	✓	✓

10. Building from Source

Instructions for developers and security teams who need to build from source.

10.1 Prerequisites

Requirement	Version	Notes
Java JDK	17+	Bundled with Burp Suite, or install separately (<code>brew install openjdk@17</code>)
Gradle	8+	Use the included wrapper (<code>./gradlew</code>) — no system install required
Git	Any	For cloning the repository

10.2 Clone and Build

```
# Clone the repository
git clone https://github.com/secretsifter/secretsifter-burp
cd secretsifter-burp

# Build the store JAR (BApp Store – no MCP server)
./gradlew shadowJar
# Output: build/libs/secretsifter-store-<version>.jar

# Build the full JAR (includes McpBridge for Claude Code integration)
./gradlew shadowJarFull
# Output: build/libs/secretsifter-mcp-<version>.jar

# Build both JARs in one step
./gradlew buildAll

# Run tests
./gradlew test
# Test report: build/reports/tests/test/index.html
```

Two JARs, two entry points:

`secretsifter-store-*.jar` — entry point

`SecretScannerExtensionStore` . No embedded HTTP listener. For BApp Store and community distribution.

`secretsifter-mcp-*.jar` — entry point `SecretScannerExtension` .

Includes McpBridge JSON-RPC server on `127.0.0.1:8765` for Claude Code / MCP agent integration.

10.3 Dependency Overview

Dependency	Scope	Purpose
<code>net.portswigger.burp.extensions:montoya-api</code>	compileOnly / testImplementation	Burp Montoya API — provided by Burp at runtime; not bundled in the JAR
<code>com.google.code.gson:gson:2.10.1</code>	implementation	JSON body walking — Apache 2.0
<code>org.junit.jupiter:junit-jupiter:5.10.2</code>	testImplementation	JUnit 5 — test only, not bundled

11. Credits & License

Attribution, open-source notices, and licensing terms.

11.1 License

SecretSifter is released under the **MIT License**.

MIT License

Copyright (c) 2024–2026 Hemanth Gorijala

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

11.2 Third-Party Notices

Vendor token format specifications are publicly documented by their respective service providers (GitHub, GitLab, AWS, Stripe, OpenAI, Slack, and others). The regex patterns implementing those formats are the author's independent work.

The extension bundles **Gson 2.10.1** (Google Inc., Apache License 2.0) for JSON body walking. All other dependencies are provided by Burp at runtime and are not bundled.

11.3 Author

Field	Detail
Author	Hemanth Gorijala
ORCID	0009-0006-9810-4001
Repository	https://github.com/secretsifter/secretsifter-burp
License	MIT
BApp Store	Secret Sifter (PortSwigger BApp Store)